Module 03b — OOP Part 2: Inheritance & Composition

Phase: Advanced Python
Previous module: OOP Part 1 — Classes & Encapsulation
Next module: OOP Part 3 — Polymorphism & Project

Series: OOP overview · Part 1 · Part 2 (this file) · Part 3 · Part 4

Assignment (take-home): Still the same portfolio deliverable — **Mini-Project (Assignment): BankAccount OOP CLI** in Part 3. Today's homework is inheritance drills; stretch = `SavingsAccount` toward that push.

Lab kit: 15_Lab_Assets/17_bank_account_oop/

Teach order today: Inheritance basics → **types of inheritance (with diagrams)** → overriding → then HAS-A / composition as the alternative.

Motivation — Feel the pain of copy-paste "almost the same" classes

Without inheritance, specialised accounts often become copy-paste:

```
class BankAccount:
    def __init__(self, holder, balance=0):
        self.holder = holder
        self._balance = float(balance)

    def deposit(self, amount):
        if amount <= 0:
            print("Invalid amount")
            return
        self._balance += amount

    def get_balance(self):
        return self._balance

class SavingsAccountCopyPaste:
    def __init__(self, holder, balance=0, interest_rate=0.04):
        self.holder = holder
        self._balance = float(balance)
        self.interest_rate = interest_rate

    def deposit(self, amount):          # duplicated
        if amount <= 0:
            print("Invalid amount")
            return
        self._balance += amount

    def get_balance(self):              # duplicated
        return self._balance

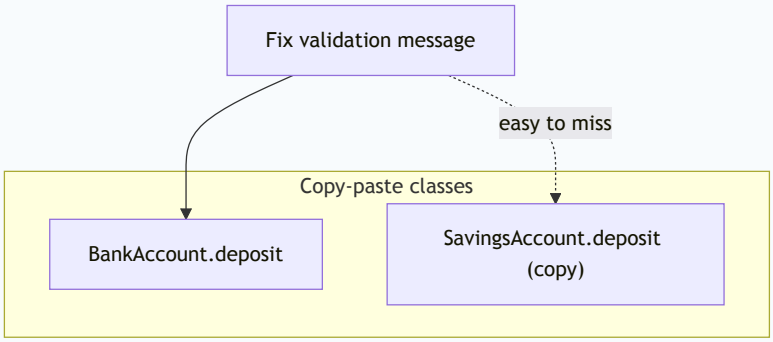
    def apply_interest(self):
        self.deposit(self._balance * self.interest_rate)
```

Problems:

1. Deposit / balance rules are written **twice**.

2. Fix a bug in `BankAccount.deposit` → easy to forget the copy in Savings.
3. "Savings **is a** bank account" is a real domain relationship — the code should say so.

Visual — duplicated behaviour vs one parent:



Key takeaways:

- Same deposit rules in two classes = bug factory
- Inheritance = child **reuses** parent and adds only what is new
- Syntax taste: `class SavingsAccount(BankAccount):`

With inheritance, Savings **reuses** BankAccount and only adds what is new:

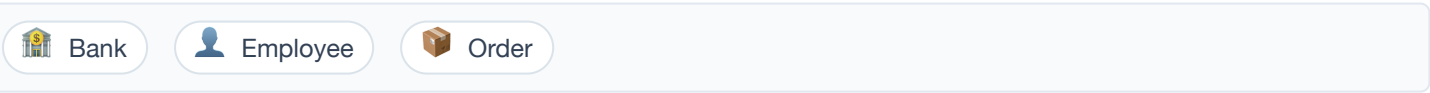
```
class SavingsAccount(BankAccount):
    def __init__(self, holder, balance=0, interest_rate=0.04):
        super().__init__(holder, balance)
        self.interest_rate = interest_rate

    def apply_interest(self):
        interest = self._balance * self.interest_rate
        self.deposit(interest)
        print(f"Interest ₹{interest:.2f} added")
```

Important Concepts

1. Inheritance basics — IS-A, parent / child, `super()`

Real-world analogy: A **smartphone IS-A phone**. It still makes calls (parent behaviour). It adds camera and apps (child extras). You do not rebuild dialling from scratch. `super()` is "first set up the phone part, then add the smartphone part." **SecureBank** `SavingsAccount` **is a** `BankAccount`. **AcmeHR** `Manager` **is an** `Employee`. **MedCare** `Inpatient` **is a** `Patient`.



Where used? SecureBank `SavingsAccount` is a `BankAccount`; **AcmeHR** `Manager` is an `Employee`; `Car` is a `Vehicle`.

Definition: Inheritance lets a **child class** reuse and extend a **parent class**.
In plain English: the child **IS-A** specialised kind of the parent.

Purpose: Write shared behaviour once on the parent; children only add / change what is special.

Word	Meaning
Parent (base / super class)	The general class — `BankAccount`, `Person`
Child (derived / sub class)	The specialised class — `SavingsAccount`, `Employee`
IS-A	`SavingsAccount` **is a** `BankAccount`

Word	Meaning
`super()`	Call the parent's method / `__init__` from the child

```
class Person:
    def __init__(self, name):
        self.name = name

    def greet(self):
        return f"Hi, I am {self.name}"

class Employee(Person):                 # Employee IS-A Person
    def __init__(self, name, emp_id):
        super().__init__(name)          # init the parent part first
        self.emp_id = emp_id

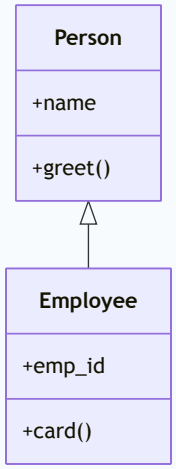
    def card(self):
        return f"{self.greet()} ({self.emp_id})"

e = Employee("Priya", 501)
print(e.card())
print("name from parent:", e.name)
```

Expected Output:

```
Hi, I am Priya (501)
name from parent: Priya
```

Visual — child gets parent features:



Key takeaways:

- `class Child(Parent):` = inheritance
- Child **IS-A** Parent
- Call `super().__init__(...)` when the parent owns required fields

Common Error: forgetting `super().__init__(...)` → later code crashes looking for `self.name` / `self._balance` from the parent (missing attribute).

Note:

- Always initialise the parent before adding child-only fields.
- Inheritance is for **specialisation**, not for "I want to reuse one helper method" (a plain function or composition may be enough).

Try these (3 project examples):

Program 1 — SecureBank SavingsAccount inherits BankAccount

```
class BankAccount:
    def __init__(self, holder, balance=0):
        self.holder = holder
        self._balance = float(balance)

    def deposit(self, amount):
        if amount <= 0:
            print("Invalid amount")
            return
        self._balance += amount

    def get_balance(self):
        return self._balance

class SavingsAccount(BankAccount):
    def __init__(self, holder, balance=0, interest_rate=0.04):
        super().__init__(holder, balance)
        self.interest_rate = interest_rate

    def apply_interest(self):
        interest = self._balance * self.interest_rate
        self.deposit(interest)
        print(
            f"Interest ₹{interest:.2f} added. "
            f"Balance: ₹{self.get_balance():.2f}"
        )

acct = SavingsAccount("Priya", 10000, 0.05)
print("Before:", acct.get_balance())
acct.apply_interest()
print("Holder still:", acct.holder)
```

Expected Output:

```
Before: 10000.0
Interest ₹500.00 added. Balance: ₹10500.00
Holder still: Priya
```

Program 2 — AcmeHR Intern is an Employee

```
class Employee:
    def __init__(self, name, emp_id):
        self.name = name
        self.emp_id = emp_id

    def badge(self):
        return f"{self.name} ({self.emp_id})"

class Intern(Employee):
    def __init__(self, name, emp_id, college):
        super().__init__(name, emp_id)
        self.college = college

    def badge(self):
        return f"{super().badge()} — intern from {self.college}"
```

```
print(Intern("Anita", 801, "KSR").badge())
print(Intern("Ravi", 802, "NIT Trichy").badge())
```

Expected Output:

```
Anita (801) — intern from KSR
Ravi (802) — intern from NIT Trichy
```

Program 3 — MedCare Inpatient is a Patient

```
class Patient:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def card(self):
        return f"{self.name}, {self.age}y"

class Inpatient(Patient):
    def __init__(self, name, age, ward):
        super().__init__(name, age)
        self.ward = ward

    def card(self):
        return f"{super().card()}, ward {self.ward}"

print(Inpatient("Meera Iyer", 54, "ICU").card())
print("IS-A Patient?", isinstance(Inpatient("Meera Iyer", 54, "ICU"), Patient))
```

Expected Output:

```
Meera Iyer, 54y, ward ICU
IS-A Patient? True
```

---

2. Types of inheritance (with diagrams)

Real-world analogy: Family-tree **shapes**, not extra magic. **Single:** one parent, one child — savings is a bank account. **Multilevel:** grandparent → parent → child — Person → Employee → Manager. **Hierarchical:** one parent, many children — Vehicle → Car and Bike; Account → Savings and Current. **Multiple:** one child, two parents — “driver + cook” on one person; useful, easy to abuse. Real products mostly use **single** and **hierarchical**. **SecureBank**, **AcmeHR**, and **TechMart** almost never need a spaghetti of five parents.



Where used? Interviews always ask these shapes; real apps use single / hierarchical most often.

Definition: How many parents / children you wire — the **shape** of the tree.

Purpose: So you can read diagrams, name the type in interviews, and pick a shallow design.

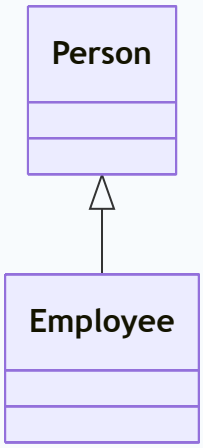
Type	Idea	Syntax taste
Single	One parent → one child	`class B(A)`

Type	Idea	Syntax taste
Multilevel	Chain: grandparent → parent → child	`class C(B)` and `class B(A)`
Hierarchical	One parent → many children	`class Car(Vehicle)`, `class Bike(Vehicle)`
Multiple	Many parents → one child	`class C(A, B)` — supported; prefer composition when unsure

---

2a. Single inheritance

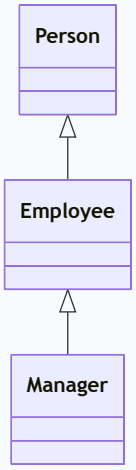
One parent, one child — the everyday default.



Key takeaways:
- One arrow up to one parent = **single**
- Everyday default shape

2b. Multilevel inheritance

A chain: child inherits from parent, parent inherits from grandparent.



Key takeaways:
- A **chain** of IS-A links = **multilevel**
- Keep chains short (2–3 levels)

```
class Person:
    def role(self):
        return "Person"

class Employee(Person):
    def role(self):
        return "Employee"

class Manager(Employee):          # multilevel: Manager → Employee → Person
    def role(self):
```

```
        return "Manager"

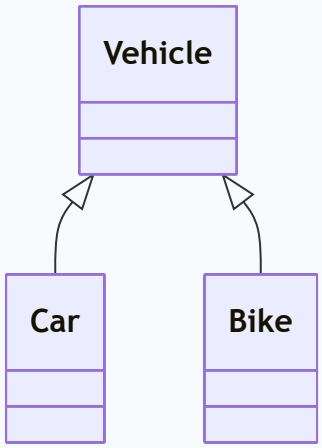
m = Manager()
print(m.role())
print("Manager is Person?", isinstance(m, Person))
print("MRO:", [c.__name__ for c in Manager.mro()])
```

Expected Output:

```
Manager
Manager is Person? True
MRO: ['Manager', 'Employee', 'Person', 'object']
```

2c. Hierarchical inheritance

One parent, **many** specialised children — great prep for polymorphism (Part 3).



Key takeaways:

- One parent, many children = **hierarchical**
- Great setup for polymorphism later (Part 3)

```
class Vehicle:
    def __init__(self, brand):
        self.brand = brand

    def start(self):
        return f"{self.brand} starting…"

class Car(Vehicle):
    def info(self):
        return f"Car: {self.brand}"

class Bike(Vehicle):
    def info(self):
        return f"Bike: {self.brand}"

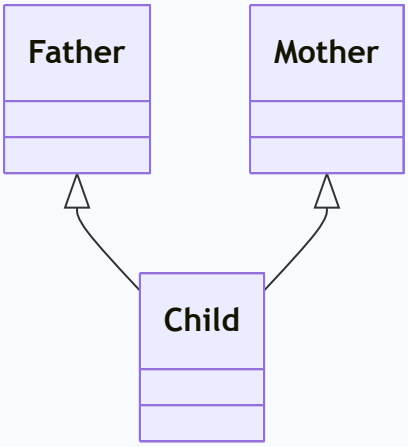
for v in [Car("Hyundai"), Bike("Royal Enfield")]:
    print(v.info(), "|", v.start())
```

Expected Output:

```
Car: Hyundai | Hyundai starting…
Bike: Royal Enfield | Royal Enfield starting…
```

2d. Multiple inheritance

One child, **two or more** parents. Python allows it; day-to-day apps often prefer composition instead.



Key takeaways:
- Many parents → one child = **multiple**
- Method name clash → MRO decides (left parent first)
- Prefer HAS-A / composition when you only need to *use* two helpers

```
class Father:
    def skills(self):
        return ["driving"]

class Mother:
    def skills(self):
        return ["cooking"]

class Child(Father, Mother):     # multiple inheritance
    def all_skills(self):
        # MRO: Child → Father → Mother → object (left parent first)
        return Father.skills(self) + Mother.skills(self)

c = Child()
print(c.all_skills())
print("MRO:", [x.__name__ for x in Child.mro()])
print("Which skills() wins alone?", c.skills())  # Father first in MRO
```

Expected Output:

```
['driving', 'cooking']
MRO: ['Child', 'Father', 'Mother', 'object']
Which skills() wins alone? ['driving']
```

Multiple inheritance note: If both parents define the same method, Python uses **MRO** (Method Resolution Order) — roughly "child first, then left-to-right parents." Inspect with `ClassName.mro()`. You do not need the C3 algorithm for this course; know that order matters: `class C(A, B)` vs `class C(B, A)`.

Common Error: using multiple inheritance just to "mix features" when `self.engine = Engine()` would be clearer.

Try these (3 project examples):

Program 1 — SecureBank hierarchical accounts under BankAccount

```
class BankAccount:
    def __init__(self, holder, balance=0):
        self.holder = holder
        self._balance = float(balance)
```

```
    def get_balance(self):
        return self._balance

class SavingsAccount(BankAccount):
    def kind(self):
        return "savings"

class CurrentAccount(BankAccount):
    def kind(self):
        return "current"

for acct in [SavingsAccount("Priya", 1000), CurrentAccount("Ravi", 500)]:
    print(acct.holder, acct.kind(), acct.get_balance())
```

Expected Output:

```
Priya savings 1000.0
Ravi current 500.0
```

Program 2 — AcmeHR multilevel: Person → Employee → Manager

```
class Person:
    def __init__(self, name):
        self.name = name

    def label(self):
        return self.name

class Employee(Person):
    def __init__(self, name, emp_id):
        super().__init__(name)
        self.emp_id = emp_id

    def label(self):
        return f"{super().label()} ({self.emp_id})"

class Manager(Employee):
    def __init__(self, name, emp_id, team_size):
        super().__init__(name, emp_id)
        self.team_size = team_size

    def label(self):
        return f"{super().label()} — team of {self.team_size}"

print(Manager("Meera", 900, 6).label())
```

Expected Output:

```
Meera (900) — team of 6
```

Program 3 — TechMart hierarchical catalogue (Product → Grocery / Electronics)

```
class Product:
    def __init__(self, sku, price):
        self.sku = sku
        self.price = price
```

```
    def tag(self):
        return f"{self.sku} ₹{self.price}"

class Grocery(Product):
    def tag(self):
        return f"GROCERY {super().tag()}"

class Electronics(Product):
    def tag(self):
        return f"ELECTRONICS {super().tag()}"

for p in [Grocery("ATTA-5KG", 280), Electronics("USB-C", 199)]:
    print(p.tag())
```

Expected Output:

```
GROCERY ATTA-5KG ₹280
ELECTRONICS USB-C ₹199
```

---

3. Method overriding

Real-world analogy: Company **leave policy**. The default form says "12 days." Managers fill the *same form* but write "18 days." Same question (`leaves()` / `net()` / `info()`), different answer. You still *are* an employee — you just specialise the rule. **AcmeHR** overrides `payslip()`. **SecureBank** overrides `withdraw()` for a current-account overdraft. **TechMart** overrides `ship()` for COD vs prepaid.

Definition: The child defines a method with the **same name** as the parent, with a new body.
Purpose: Specialise behaviour while keeping the same call site (`obj.info()`, `obj.net_salary()`).

```
class Vehicle:
    def __init__(self, brand, year):
        self.brand = brand
        self.year = year

    def info(self):
        return f"{self.brand} ({self.year})"

class Car(Vehicle):
    def __init__(self, brand, year, doors):
        super().__init__(brand, year)
        self.doors = doors

    def info(self):   # override
        return f"Car: {super().info()}, doors={self.doors}"

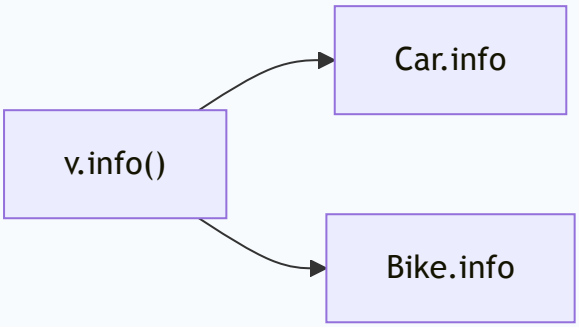
class Bike(Vehicle):
    def info(self):   # override
        return f"Bike: {super().info()}"

for v in [Car("Hyundai", 2022, 4), Bike("Royal Enfield", 2021)]:
    print(v.info())
```

Expected Output:

```
Car: Hyundai (2022), doors=4
Bike: Royal Enfield (2021)
```

Visual — same method name, different body:



Key takeaways:

- Override = same method name in child, new body
- Call `super().method()` when you still want parent behaviour plus extras
- Hierarchical + override = natural setup for Part 3 polymorphism

Common Error: overriding `__init__` or `info` and forgetting `super()` when the parent still owns required setup.

Try these (3 project examples):

Program 1 — AcmeHR override payslip on Manager

```
class Employee:
    def __init__(self, name, basic):
        self.name = name
        self.basic = basic

    def net(self):
        return self.basic * 0.9

    def payslip(self):
        return f"{self.name} net ₹{self.net():.0f}"

class Manager(Employee):
    def __init__(self, name, basic, bonus):
        super().__init__(name, basic)
        self.bonus = bonus

    def net(self):
        return super().net() + self.bonus

    def payslip(self):
        return f"{self.name} (Mgr) net ₹{self.net():.0f}"

print(Employee("Ravi", 30000).payslip())
print(Manager("Meera", 30000, 8000).payslip())
```

Expected Output:

```
Ravi net ₹27000
Meera (Mgr) net ₹35000
```

Program 2 — SecureBank override withdraw (current allows overdraft)

```
class BankAccount:
    def __init__(self, holder, balance):
        self.holder = holder
        self._balance = float(balance)

    def withdraw(self, amount):
        if amount > self._balance:
            print("Insufficient funds")
            return
        self._balance -= amount

    def get_balance(self):
        return self._balance

class CurrentAccount(BankAccount):
    def withdraw(self, amount):
        # overdraft up to ₹1000
        if amount > self._balance + 1000:
            print("Overdraft limit exceeded")
            return
        self._balance -= amount

s = BankAccount("Priya", 500)
c = CurrentAccount("Ravi", 500)
s.withdraw(800)   # blocked
c.withdraw(800)   # allowed — overdraft
print("Priya:", s.get_balance())
print("Ravi:", c.get_balance())
```

Expected Output:

```
Insufficient funds
Priya: 500.0
Ravi: -300.0
```

Program 3 — TechMart override ship() for COD vs prepaid

```
class Order:
    def ship(self):
        return "Packed — prepaid, dispatch today"

class CodOrder(Order):
    def ship(self):
        return "Packed — collect cash, then dispatch"

print(Order().ship())
print(CodOrder().ship())
```

Expected Output:

```
Packed — prepaid, dispatch today
Packed — collect cash, then dispatch
```

---

4. HAS-A (composition) — when *not* to inherit

Real-world analogy: A college **HAS** departments. A student **HAS** an ID card. The student is **not** an ID card — so do not inherit `IdCard`. A car **HAS** an engine; a car is not an engine. Inheritance is "is a specialised kind of." Composition is "owns / uses a collaborator." **AcmeHR** `Employee` has a `Laptop`. **TechMart** `OrderService` has a notifier. **MedCare** `Appointment` has a `Doctor` and a `Patient`.



Where used? AcmeHR `Employee` **has a** `Laptop`; **TechMart** `OrderService` **has a** notifier; `Car` **has an** `Engine`.

Teach this **after** inheritance types: first learn IS-A trees, then learn when a tree is the wrong tool.

Definition: Composition / HAS-A = class A *stores and uses* an instance of B (`self.laptop = Laptop(...)`).

Purpose: You need collaboration, not specialisation. A car is **not** an engine; an employee is **not** a laptop.

Rule of thumb (decide after you can sketch inheritance types):

Need	Choose
Child **is a** specialised parent	**IS-A** inheritance
Object only **uses** / owns another object	**HAS-A** composition

```
class Laptop:
    def __init__(self, model):
        self.model = model

    def spec(self):
        return f"Laptop: {self.model}"

class Person:
    def __init__(self, name):
        self.name = name

    def greet(self):
        return f"Hi, I am {self.name}"

class Employee(Person):                    # IS-A Person (inheritance)
    def __init__(self, name, emp_id, laptop):
        super().__init__(name)
        self.emp_id = emp_id
        self.laptop = laptop               # HAS-A Laptop (composition)

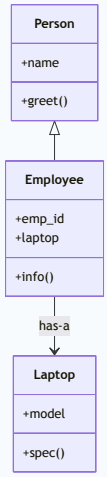
    def info(self):
        return f"{self.greet()} ({self.emp_id}) — {self.laptop.spec()}"

e = Employee("Priya", 501, Laptop("Dell Latitude"))
print(e.info())
print("isinstance Employee→Person?", isinstance(e, Person))
print("isinstance Employee→Laptop?", isinstance(e, Laptop))
```

Expected Output:

```
Hi, I am Priya (501) — Laptop: Dell Latitude
isinstance Employee→Person? True
isinstance Employee→Laptop? False
```

Visual — own a collaborator, do not inherit it:



Key takeaways:

- Inheritance = IS-A
- Composition = HAS-A (`self.other = Other(...)`)
- `isinstance(e, Laptop)` is False — laptop is owned, not inherited
- Prefer composition when unsure / when mixing unrelated features

Note:

- Deep inheritance trees get brittle — prefer composition when unsure.
- Composition keeps parts swappable (`SmsNotifier` vs `EmailNotifier`) without a messy parent list.
- Richer design patterns live in optional Professional Python Deep Dive.

Try these (3 project examples):

Program 1 — TechMart OrderService HAS-A notifier

```
class SmsNotifier:
    def send(self, message):
        print(f"SMS: {message}")

class EmailNotifier:
    def send(self, message):
        print(f"EMAIL: {message}")

class OrderService:
    def __init__(self, notifier):
        self.notifier = notifier   # HAS-A

    def place(self, order_id):
        self.notifier.send(f"Order {order_id} placed")

OrderService(SmsNotifier()).place("ORD-101")
OrderService(EmailNotifier()).place("ORD-102")
```

Expected Output:

```
SMS: Order ORD-101 placed
EMAIL: Order ORD-102 placed
```

Program 2 — AcmeHR Employee HAS-A Laptop (not IS-A Laptop)

```
class Laptop:
    def __init__(self, model):
        self.model = model

    def spec(self):
        return f"Laptop {self.model}"

class Employee:
    def __init__(self, name, laptop):
        self.name = name
        self.laptop = laptop   # HAS-A

    def desk(self):
        return f"{self.name} uses {self.laptop.spec()}"

print(Employee("Priya", Laptop("Dell Latitude")).desk())
print("Employee IS-A Laptop?", isinstance(Employee("Priya", Laptop("X")), Laptop))
```

Expected Output:

```
Priya uses Laptop Dell Latitude
Employee IS-A Laptop? False
```

Program 3 — MedCare Appointment HAS-A Doctor and Patient

```
class Person:
    def __init__(self, name):
        self.name = name

class Doctor(Person):
    pass

class Patient(Person):
    pass

class Appointment:
    def __init__(self, doctor, patient, slot):
        self.doctor = doctor     # HAS-A
        self.patient = patient   # HAS-A
        self.slot = slot

    def slip(self):
        return f"{self.patient.name} → Dr {self.doctor.name} @ {self.slot}"

print(Appointment(Doctor("Meera Iyer"), Patient("Anita"), "10:30").slip())
```

Expected Output:

```
Anita → Dr Meera Iyer @ 10:30
```

---

5. `isinstance` and when *not* to switch on types

Real-world analogy: Campus **security** asks “Are you staff?” A professor **is** staff (subclass counts). A visitor is not. `isinstance` is that gate question. `type(obj) is Staff` is the rude version that rejects professors because they are a *kind* of staff. Use the gate for **guards** (don’t print a balance on a dict). Do **not** build a giant “if savings elif current elif wallet” tree — that is Part 3 polymorphism’s job. **SecureBank** rejects a raw dict in `print_balance`. **MedCare** accepts `Inpatient` where `Patient` is required.

Definition: `isinstance(obj, Class)` asks “is this object this type (or a subclass)?”
Purpose: Useful for guards — dangerous as a giant `if/elif` tree that recreates the problem polymorphism solves (Part 3).

```
class Animal:
    pass

class Dog(Animal):
    def speak(self):
        return "woof"

class Cat(Animal):
    def speak(self):
        return "meow"

def describe(creature):
    if not isinstance(creature, Animal):
        print("Not an Animal")
        return
    print(creature.speak())

describe(Dog())
describe(Cat())
describe("not an animal")
```

Expected Output:

```
woof
meow
Not an Animal
```

Prefer this (Part 3 style) over type switches for behaviour:

```
for creature in [Dog(), Cat()]:
    print(creature.speak())   # same call, different behaviour
```

Note:

- `type(obj) is Dog` rejects subclasses; `isinstance(obj, Dog)` accepts them — usually what you want for IS-A checks.
- If you find yourself writing `if isinstance(...): ... elif isinstance(...):` for business behaviour, you probably want override / duck typing instead.

Try these (3 project examples):

Program 1 — SecureBank: reject a dict, accept a real account

```
class BankAccount:
    def __init__(self, balance):
        self._balance = balance

    def get_balance(self):
        return self._balance

def print_balance(account):
    if not isinstance(account, BankAccount):
        print("Need a BankAccount")
        return
    print("Balance:", account.get_balance())

print_balance(BankAccount(500))
print_balance({"balance": 500})
```

Expected Output:

```
Balance: 500
Need a BankAccount
```

Program 2 — MedCare: Inpatient counts as Patient (`type` vs `isinstance`)

```
class Patient:
    pass

class Inpatient(Patient):
    pass

p = Inpatient()
print("type is Patient?", type(p) is Patient)            # False — too strict
print("isinstance Patient?", isinstance(p, Patient))     # True — gate accepts subclass
```

Expected Output:

```
type is Patient? False
isinstance Patient? True
```

Program 3 — TechMart: don't switch on type for shipping — call the method

```
class Order:
    def ship(self):
        return "standard dispatch"

class ExpressOrder(Order):
    def ship(self):
        return "same-day dispatch"

def dispatch(order):
    if not isinstance(order, Order):
        print("Need an Order")
        return
    print(order.ship())   # polymorphism — not if ExpressOrder elif Order
```

```
dispatch(Order())
dispatch(ExpressOrder())
dispatch("ORD-101")
```

Expected Output:

```
standard dispatch
same-day dispatch
Need an Order
```

Step-by-Step Annotated Python 3 Code

Predict output, then run.

Example 1 — Employee / Manager (single + override)

```
class Employee:
    tax_rate = 0.10

    def __init__(self, emp_id, name, basic_salary):
        self.emp_id = emp_id
        self.name = name
        self.basic_salary = basic_salary

    def net_salary(self):
        return self.basic_salary * (1 - Employee.tax_rate)

    def payslip(self):
        return f"{self.name} (Emp) net ₹{self.net_salary():.2f}"

class Manager(Employee):
    def __init__(self, emp_id, name, basic_salary, bonus=5000):
        super().__init__(emp_id, name, basic_salary)
        self.bonus = bonus

    def net_salary(self):
        return super().net_salary() + self.bonus

    def payslip(self):
        return (
            f"{self.name} (Mgr) net ₹{self.net_salary():.2f} "
            f"incl. bonus ₹{self.bonus:.2f}"
        )

staff = [
    Employee(1, "Ravi", 30000),
    Manager(2, "Meera", 30000, bonus=8000),
]
for person in staff:
    print(person.payslip())
```

Expected Output:

```
Ravi (Emp) net ₹27000.00
Meera (Mgr) net ₹35000.00 incl. bonus ₹8000.00
```

Remember: Manager **is an** Employee — shared tax rule stays on the parent; bonus is the specialisation.

Example 2 — Hierarchical: Savings vs Current

```
class BankAccount:
    def __init__(self, holder, balance=0):
        self.holder = holder
        self._balance = float(balance)

    def deposit(self, amount):
        if amount <= 0:
            print("Invalid amount")
            return False
        self._balance += amount
        return True

    def withdraw(self, amount):
        if amount <= 0:
            print("Invalid amount")
            return False
        if amount > self._balance:
            print("Insufficient funds")
            return False
        self._balance -= amount
        return True

    def get_balance(self):
        return self._balance

class SavingsAccount(BankAccount):
    def apply_interest(self, rate=0.04):
        self.deposit(self._balance * rate)

class CurrentAccount(BankAccount):
    def __init__(self, holder, balance=0, overdraft_limit=5000):
        super().__init__(holder, balance)
        self.overdraft_limit = overdraft_limit

    def withdraw(self, amount):
        if amount <= 0:
            print("Invalid amount")
            return False
        if amount > self._balance + self.overdraft_limit:
            print("Overdraft limit exceeded")
            return False
        self._balance -= amount
        return True

savings = SavingsAccount("Priya", 10000)
savings.apply_interest(0.05)
print("Savings:", savings.get_balance())

current = CurrentAccount("Ravi", 1000, overdraft_limit=2000)
```

```
current.withdraw(2500)
print("Current:", current.get_balance())
current.withdraw(1000)
```

Expected Output:

```
Savings: 10500.0
Current: -1500.0
Overdraft limit exceeded
```

Example 3 — IS-A + HAS-A together (after you know both)

```
class Car:
    def __init__(self, name, model):
        self.name = name
        self.model = model

    def getinfo(self):
        return f"{self.name} {self.model}"

class Person:
    def __init__(self, name):
        self.name = name

class Employee(Person):
    def __init__(self, name, emp_id, car):
        super().__init__(name)
        self.emp_id = emp_id
        self.car = car                # HAS-A Car

    def card(self):
        return f"{self.name} ({self.emp_id}) drives {self.car.getinfo()}"

print(Employee("Anita", 101, Car("Innova", "Crysta")).card())
print(Employee("Vikram", 102, Car("Nexon", "EV")).card())
```

Expected Output:

```
Anita (101) drives Innova Crysta
Vikram (102) drives Nexon EV
```

Practice Exercises

Intermediate

1. `Rectangle` with `area()` / `perimeter()`; `Square` inherits from `Rectangle` and only needs `side` (**single**).

   Model answer:

```
class Rectangle:
    def __init__(self, w, h):
        self.w, self.h = w, h

    def area(self):
```

```
        return self.w * self.h

    def perimeter(self):
        return 2 * (self.w + self.h)

class Square(Rectangle):
    def __init__(self, side):
        super().__init__(side, side)

sq = Square(4)
print(sq.area(), sq.perimeter())  # 16 16
```

Expected Output:

```
16 16
```

2. **Multilevel:** `Person` → `Employee` → `Developer` with `language` and a `label()` override that uses `super()`.
   Model answer:

```
class Person:
    def __init__(self, name):
        self.name = name

    def label(self):
        return self.name

class Employee(Person):
    def __init__(self, name, emp_id):
        super().__init__(name)
        self.emp_id = emp_id

    def label(self):
        return f"{super().label()} ({self.emp_id})"

class Developer(Employee):
    def __init__(self, name, emp_id, language):
        super().__init__(name, emp_id)
        self.language = language

    def label(self):
        return f"{super().label()} — {self.language}"

print(Developer("Anita", 11, "Python").label())
```

Expected Output:

```
Anita (11) — Python
```

3. Sketch (on paper or Mermaid) **hierarchical** inheritance for `Shape` → `Circle` / `Rectangle`. Then implement `area()`.
   Model answer:

```
import math

class Shape:
```

```
    def area(self):
        return 0.0

class Circle(Shape):
    def __init__(self, r):
        self.r = r

    def area(self):
        return math.pi * self.r ** 2

class Rectangle(Shape):
    def __init__(self, w, h):
        self.w, self.h = w, h

    def area(self):
        return self.w * self.h

for s in [Circle(2), Rectangle(3, 4)]:
    print(round(s.area(), 2))
```

Expected Output:

```
12.57
12
```

4. **Composition:** `Library` HAS-A list of `Book` objects; implement `add(book)` and `count()`.
   Model answer:

```
class Book:
    def __init__(self, title):
        self.title = title

class Library:
    def __init__(self, name):
        self.name = name
        self.books = []

    def add(self, book):
        self.books.append(book)

    def count(self):
        return len(self.books)

lib = Library("KSR Central")
lib.add(Book("Clean Code"))
lib.add(Book("Fluent Python"))
print(lib.name, lib.count())  # KSR Central 2
```

Expected Output:

```
KSR Central 2
```

5. Extend lab kit: add `SavingsAccount(BankAccount)` with `add_interest(rate=0.04)` that deposits `balance * rate`.
   Model answer:

```
class SavingsAccount(BankAccount):
    def add_interest(self, rate=0.04):
        self.deposit(self.get_balance() * rate)
```

6. Explain in 3–4 lines (no code): when would you **not** make `Car` inherit from `Engine`?
   Model answer:
   A car **has an** engine; it is not a kind of engine. Inheritance would expose engine internals as if they were car behaviour and break IS-A thinking. Use composition: `self.engine = Engine(...)`.

Interview bite (Part 2)

Q. Name the four inheritance types and one diagram each.
A: Single (A←B), multilevel (A←B←C), hierarchical (A←B and A←C), multiple (A→C←B).

Q. Inheritance vs composition?
A: Inheritance when the child **is a** specialised parent. Composition when an object **has a** collaborator it uses. Prefer composition when unsure.

Q. Why `super().__init__`?
A: So the parent can set up the attributes it owns before the child adds its own. Skipping it often causes missing-attribute bugs.

Q. What is MRO in one sentence?
A: The order Python searches for attributes/methods in an inheritance graph (`ClassName.mro()`).

Next

Assignment: push **Mini-Project (Assignment): BankAccount OOP CLI** in Part 3 — lab kit.

Continue with **OOP Part 3 — Polymorphism & Project**.